HW-6

Greyson Knippel
CPSC 321, Fall 2026

November 9, 2026

question 1

(a) relation, primary key, and functional dependencies

an online game store keeps track of each users game library in one table:

`library(owner_id, game_id, username, game_title, developer, hours_played)`

Each row says that the user with `owner_id` (whose display name is `username`) owns the game `game_id` (with its `game_title` and `developer`) and has played it for `hours_played` hours.

primary key: `(owner_id, game_id)`. A user can own a given game only once, so the pair identifies a row, and `owner_id, game_id` $\rightarrow$ `username, game_title, developer, hours_played`.

non key functional dependencies:

FD1: `owner_id` $\rightarrow$ `username`. Each account has one display name, so every row for the same owner must show the same username

FD2: `game_id` $\rightarrow$ `game_title, developer`. Each game in the store has one title and one developer, so every row for the same game must show the same title and developer.

both are non trivial and non key dependencies

(b) instance with redundant data

owner_id	game_id	username	game_title	developer	hours_played
101	413150	grey	Stardew Valley	ConcernedApe	212
101	367520	grey	Hollow Knight	Team Cherry	48
202	413150	pixelpat	Stardew Valley	ConcernedApe	95
303	1145360	nova	Hades	Supergiant Games	30

The instance satisfies both FDs: owner 101 is always "grey", and game 413150 is always "Stardew Valley" by ConcernedApe. Its also redundant: "owner 101 is grey" is stored twice, and "413150 is Stardew Valley by ConcernedApe" is stored twice. With a real store, a popular game owned by millions of users would have its title and developer repeated millions of times

(c) anomalies

update anomaly user 101 changes their username from "grey" to "greyson". If only the first row is updated, the second row still says "grey", so the table claims owner 101 has two different usernames. This breaks `owner_id` → `username`. To make the change correctly we have to update every row in user 101s library

insertion anomaly The store adds a new game (say `game_id` 2000000, "Starlight Drift" by a new studio) before anyone has bought it. We cannot record its title and developer, because `owner_id` is part of the primary key and cannot be NULL, and there is no owner yet. The same problem happens for a new user who hasn't bought any games: their username cant be stored

deletion anomaly user 303 refunds Hades, so their row is deleted. That row was the only one for user 303 and the only one for game 1145360, so deleting it also loses the fact that user 303 is "nova" and that Hades is made by Supergiant Games, even though we only meant to remove one purchase

question 2

(a) decomposition: if $X \to YZ$, then $X \to Y$ and $X \to Z$

step	FD	justification
1	$X \to YZ$	given
2	$YZ \to Y$	reflexivity, since $Y \subseteq YZ$
3	$X \to Y$	transitivity on (1) and (2)
4	$YZ \to Z$	reflexivity, since $Z \subseteq YZ$
5	$X \to Z$	transitivity on (1) and (4)

steps (3) and (5) give $X \to Y$ and $X \to Z$

(b) pseudotransitivity: if $X \to Y$ and $YW \to Z$, then $XW \to Z$

step	FD	justification
1	$X \to Y$	given
2	$XW \to YW$	augmentation of (1) with W
3	$YW \to Z$	given
4	$XW \to Z$	transitivity on (2) and (3)

step (4) gives $XW \to Z$

question 3

(a) $R(a, b, c, d)$, FDs $\{b \to c,\ d \to a\}$

candidate key: bd

- **determines everything:** $\{b, d\} \xrightarrow{b \to c} \{b, c, d\} \xrightarrow{d \to a} \{a, b, c, d\}$. so bd determines all of R

- **minimal:** b alone only determines $\{b, c\}$ (missing a, d) and d alone only determines $\{a, d\}$ (missing b, c), so neither b nor d alone is a superkey
- **no other keys:** b and d never appear on the right-hand side of an FD, so every superkey must contain both of them, so every superkey contains bd. any superkey other than bd is then a proper superset of bd and isnt minimal

(b) $R(a, b, c, d, e)$**, FDs** $\{a \to bc,\ cd \to e,\ b \to d,\ e \to a\}$

candidate keys: a**,** e**,** bc**,** cd

- a**:** $\{a\} \xrightarrow{a \to bc} \{a, b, c\} \xrightarrow{b \to d} \{a, b, c, d\} \xrightarrow{cd \to e} \{a, b, c, d, e\}$. minimal since it is a single attribute
- e**:** $\{e\} \xrightarrow{e \to a} \{a, e\}$, and from a we get everything from above. minimal since it is a single attribute
- bc**:** $\{b, c\} \xrightarrow{b \to d} \{b, c, d\} \xrightarrow{cd \to e} \{b, c, d, e\} \xrightarrow{e \to a} \{a, b, c, d, e\}$. minimal since b alone only determines $\{b, d\}$ and c alone only determines $\{c\}$
- cd**:** $\{c, d\} \xrightarrow{cd \to e} \{c, d, e\} \xrightarrow{e \to a} \{a, c, d, e\} \xrightarrow{a \to bc} \{a, b, c, d, e\}$. minimal since c alone only determines $\{c\}$ and d alone only determines $\{d\}$ (no FD has just c or just d on the left)

no other keys: every set of attributes falls into one of two cases:

- **it contains** a **or** e**:** then it already contains the key a or the key e. anything extra can be removed and it still works, so it isnt minimal. the only keys here are a and e themselves
- **it contains neither** a **nor** e**:** then it only uses b, c, d, so there are just 7 sets to check:

set	result
b	only determines $\{b, d\}$, not a superkey
c	only determines $\{c\}$, not a superkey
d	only determines $\{d\}$, not a superkey
bd	only determines $\{b, d\}$, not a superkey
bc	key (found above)
cd	key (found above)
bcd	contains bc, so not minimal

so the candidate keys are exactly a, e, bc, and cd

(c) $R(a, b, c, d, e)$**, FDs** $\{c \to b,\ bd \to e,\ a \to d,\ e \to a\}$

candidate keys: ac**,** cd**,** ce

c never appears on a right side, so c is in every key. on its own, c only determines $\{b, c\}$, which is not everything, so we need to add more

- ac**:** $\{a, c\} \xrightarrow{c \to b} \{a, b, c\} \xrightarrow{a \to d} \{a, b, c, d\} \xrightarrow{bd \to e} \{a, b, c, d, e\}$. minimal since a alone only determines $\{a, d\}$ and c alone only determines $\{b, c\}$

- *cd*: $\{c, d\} \xrightarrow{c \to b} \{b, c, d\} \xrightarrow{bd \to e} \{b, c, d, e\} \xrightarrow{e \to a} \{a, b, c, d, e\}$. minimal since c alone only determines $\{b, c\}$ and d alone only determines $\{d\}$
- *ce*: $\{c, e\} \xrightarrow{c \to b} \{b, c, e\} \xrightarrow{e \to a} \{a, b, c, e\} \xrightarrow{a \to d} \{a, b, c, d, e\}$. minimal since c alone only determines $\{b, c\}$ and e alone only determines $\{a, d, e\}$

no other keys: every key must contain c, because nothing determines c. so every set we need to check has c, and falls into one of two cases:

- **it also contains** a**,** d**, or** e**:** then it already contains the key ac, cd, or ce. anything extra can be removed and it still works, so it isnt minimal. the only keys here are ac, cd, and ce themselves
- **it contains none of** a**,** d**,** e**:** then the set is just c or bc. both only determine $\{b, c\}$, so neither is a superkey

so the candidate keys are exactly ac, cd, and ce

question 4

(a) `movie_info'(award_type, award_year, director, studio_loc)`

FDs in this table `title` and `studio` are not in this table, so:

- `award_type, award_year` → `title, director` becomes `award_type, award_year` → `director` (no `title` column)
- `director` → `studio` and `studio` → `studio_loc` give `director` → `studio_loc` by transitivity (no `studio` column)

candidate key: `(award_type, award_year)`

- **determines everything:** {`award_type, award_year`} → adds `director` → adds `studio_loc`, which is every column
- **minimal:** `award_type` alone or `award_year` alone determines nothing else, since the FD needs both
- **no other keys:** `award_type` and `award_year` never appear on a right side, so every key must contain both. those two already work, so anything bigger isnt minimal

highest normal form: 2NF

- **2NF:** the key is `(award_type, award_year)`, and neither `award_type` alone nor `award_year` alone determines anything, so no column depends on only part of the key
- **not 3NF:** `director` → `studio_loc`. `director` is not a superkey and `studio_loc` is not part of a key, so this is a transitive dependency (`studio_loc` depends on the key only through `director`). so that means its not in 3NF

(b) `movie_info'(title, director, studio, studio_loc)`

FDs in this table. `award_type` and `award_year` are not in this table, so:

- `award_type, award_year` → `title, director` is gone, since its left side isnt in this table
- `director` → `studio` and `studio` → `studio_loc` are kept, and together they give `director` → `studio_loc` by transitivity

no FD has `title` on the left, so `title` determines nothing

candidate key: `(title, director)`

- **determines everything:** {`title, director`} → adds `studio` → adds `studio_loc`, which is every column
- **minimal:** `title` alone determines nothing else, and `director` alone only determines {`director, studio, studio_loc`}, which is missing `title`
- **no other keys:** `title` and `director` never appear on a right side in this table, so every key must contain both. those two already work, so anything bigger isnt minimal

highest normal form: 1NF

- **not 2NF:** `director` → `studio`. `director` is only part of the key, and `studio` is not part of a key, so a non key column depends on part of the key (a partial dependency). so that means its not in 2NF

(c) `movie_info'(award_type, award_year, studio_loc)`

FDs in this table: `title`, `director`, and `studio` are not in this table. chaining the FDs through them gives `award_type, award_year` → `director` → `studio` → `studio_loc`, so the only FD left is `award_type, award_year` → `studio_loc`. no FD has `studio_loc` on the left, so it determines nothing

candidate key: `(award_type, award_year)`

- **determines everything:** {`award_type, award_year`} → adds `studio_loc`, which is every column
- **minimal:** `award_type` alone or `award_year` alone determines nothing else, since the FD needs both
- **no other keys:** `award_type` and `award_year` never appear on a right side, so every key must contain both. those two already work, so anything bigger isnt minimal

highest normal form: BCNF

- the only FD is `award_type, award_year` → `studio_loc`, and its left side is the whole key, so it is a superkey. every FD has a superkey on the left, so the table is in BCNF (and so also 2NF and 3NF)

(d) $R(a, b, c, d, e)$ from 3(b)

from 3(b), the FDs are $\{a \to bc,\ cd \to e,\ b \to d,\ e \to a\}$ and the candidate keys are a, e, bc, and cd

highest normal form: 3NF

- **2NF and 3NF:** every column is part of some key (a and e are keys, b is in bc, c is in bc and cd, d is in cd), so there are no non key columns. 2NF and 3NF only restrict non key columns, so both hold. for 3NF, every FD's right side is part of a key
- **not BCNF:** $a \to bc$, $cd \to e$, and $e \to a$ all have a key on the left, but $b \to d$ does not, since b alone only determines $\{b, d\}$. 3NF allows it because d is part of the key cd, but BCNF needs every left side to be a superkey. so that means its not in BCNF

question 5

(a) is the decomposition lossless?

yes. the only column $R_1(a, b, c)$ and $R_2(a, d, e)$ share is a. since $a \to bc$, a determines a, b, and c, which is every column of R_1, so a is a key of R_1. a split is lossless when the shared columns are a key of one of the two tables, so this one is lossless

this works because each a value shows up in only one row of R_1, so when we join on a, every row of R_2 matches exactly one row of R_1 and then no fake rows get made

(b) is the decomposition dependency preserving?

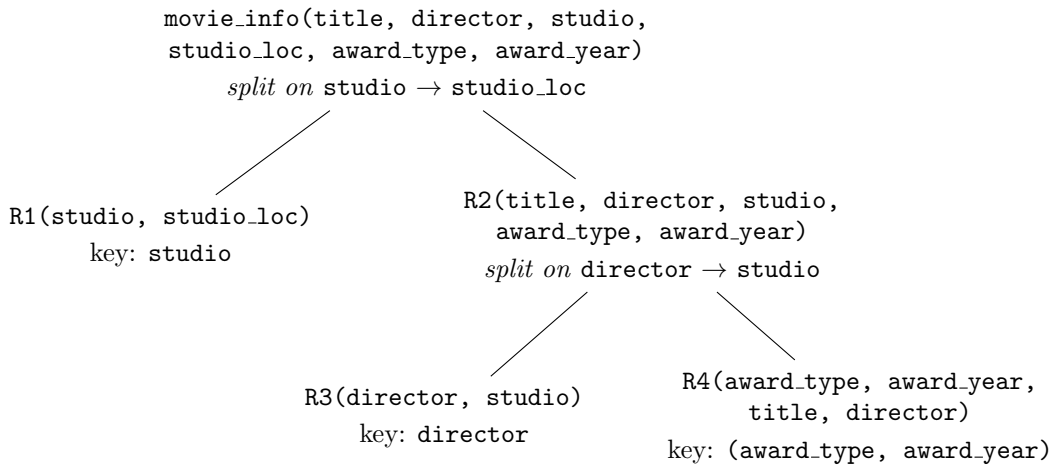
no. for it to be dependency preserving, every FD has to be checkable inside one table. checking each FD:

- $a \to bc$: a, b, c are all in R_1, so it can be checked
- $e \to a$: a, e are both in R_2, so it can be checked
- $b \to d$: b is in R_1 but d is in R_2, so it is split
- $cd \to e$: c is in R_1 but d and e are in R_2, so it is split

the kept FDs cant rebuild $b \to d$ either. in R_1, b alone determines nothing else, and R_2 doesnt have b at all. so to check $b \to d$ (or $cd \to e$) we would have to join R_1 and R_2 back together, so the decomposition is not dependency preserving

question 6

the FDs are `award_type, award_year` → `title, director`, `studio` → `studio_loc`, and `director` → `studio`. the key of `movie_info` is `(award_type, award_year)`, so `studio` → `studio_loc` and `director` → `studio` break BCNF since their left sides arent keys. for each split on $X \to Y$, the left table gets the columns of the FD and the right table gets everything except Y.



final tables: `R1(studio, studio_loc)`, `R3(director, studio)`, and `R4(award_type, award_year, title, director)`

- **BCNF:** each final table has one FD, and its left side is that tables key: `studio` → `studio_loc` in R1, `director` → `studio` in R3, and `award_type, award_year` → `title, director` in R4. so every FD has a superkey on the left
- **lossless:** in split 1 the shared column is `studio`, which is the key of R1. in split 2 the shared column is `director`, which is the key of R3. each split shares a key of one side, so the whole decomposition is lossless
- **dependency preserving:** every original FD fits inside one final table (R1, R3, and R4 above), so each one can be checked without a join. we split on `studio` → `studio_loc` first because if we split on `director` → `studio` first, `studio` and `studio_loc` would end up in different tables and that FD would be lost